Final Project

Improving TS-RAG using Maximal Marginal Relevance

Zoe Rochelle

Intelligent Systems

April 29th, 2026

Research problem and objectives

My project goal is to find some improvement that I can make to the TS-RAG repo code:
https://github.com/UConn-DSIS/TS-RAG/tree/main

I began by researching the weakness of TS-RAG (via Google) and came across the following:

TS-RAG can face

- Limited Performance Gains
- Ambiguous Value vs. Long Context
- High Inference Costs
- Long Context
- **Data Quality Dependency**

Of this list, I selected the one that looked the most interesting: **Data Quality** - I work in data engineering.

How to improve data quality?

I then researched, "ways to improve data quality of TS-RAG/RAG", and I came across problem of **redundant retrieval** [1].

Why is redundant retrieval a problem?

If the model sees the same pattern over and over, it **doesn't learn anything new from retrieval**.

Therefore, the retrieval step becomes less useful, which can limit model performance.

Potential impact of redundant data:

1. Less useful context leads to weaker predictions
2. Over bias toward one pattern
3. May miss important patterns
4. Reduces the impact of the retrieval step

Solution to redundant retrieval in TS-RAG

Diversity aware retrieval is a potential solution to redundant retrieval.

This is a strategy in which the retrieval system tries to **avoid picking results** that are too similar to each other. By 'results', I am referring to the time-series segments - aka past data.

Normal retrieval:

"What are the most similar past patterns?"
- searches for similarity only (using typical top-k retrieval)

Diversity aware retrieval:

"What are the most similar past patterns, but also not just copies of each other?"
- balances similarity and diversity

Time-series patterns: brief review

A time-series segment might look like this (electrical or traffic data):

10 → 12 → 15 → 14 → 18

The pattern is:

going up, slight dip, then going up again

Another example:

30 → 28 → 25 → 23 → 20

The pattern is:

steadily going down

When we say that, "***diversity aware retrieval*** *avoids redundant results*", we mean that it avoids retrieving too many past time-series segments that show basically the same trend or shape: increasing, decreasing, repeating, or spiking behavior.

How the original code handles time-series patterns

For each input time-series window, TS-RAG searches a database and finds the **closest matching** historical windows.

For example

An input pattern may look like this:

up, up, down, up

Retrieval might return (window = 4):

1. **up, up, down, up**
2. **up, up, down, up**
3. **up, up, down, up**
4. up, up, down, slightly up

Notice the similar pattern. This window of data is useful because they match the current input, but it does have redundancy.

Proposed change:
Maximal Marginal Relevance (MMR)

MMR is a diversity aware retrieval method – one of many.

It works as follows: for each input time-series window, TS-RAG will search a database to find the closest matching historical windows (same as before), **that are also different from the ones already selected**.

For example

If we have an input pattern like this:

up, up, down, up

MMR might return (window = 4):

1. **up, up, down, up**
2. up, down, down, up
3. up, up, flat, up
4. up, up, down, strong up

Note: that these patterns are still related to the input, but they are not all duplicates, unlike the previous slide.

Essentially: Original TS-RAG retrieves the closest past patterns, while MMR retrieves patterns that are both close and diverse.

MMR: details

So, MMR tries to balance two things:

1. **Relevance**: Is this retrieved pattern useful for the current input?
2. **Diversity**: Is this retrieved pattern different from the patterns already chosen?

Why did I choose MMR over other diversity aware retrieval methods?

- Widely used and well established method
- Simple to implement (no retraining required)
- Works with the existing retrieval pipeline (FAISS)

Note: MMR does not change how the time series segments are converted into vectors, and it does not retrain the model that creates those vectors - it only **reorders** results. This makes it low risk and computationally efficient compared to other approaches.

What should we expect to change?

Using MMR can produce different **MAE/MSE** results, but an improvement is not guaranteed.

This is because while MMR changes what gets retrieved, the forecast still depends on whether those new retrieved patterns are **actually useful**.

- Perhaps the original retrieved patterns were already good enough or the more “diverse” patterns are less similar and therefore less predictive.

A valid finding for this experiment would be if MMR changed the **retrieved time-series segments**, but the effect on MAE/MSE were small or mixed (spoiler alert).

Examining the output log

```
Build index with database: ['ETTh1_hour_512.pkl']
load database: ETTh1_hour_512.pkl
  0%|
Original indices: [7680 7800 8040 8016 8256 7728 7968 8136 8064 8160]
MMR indices: [7680 7800 8040 7728 8256 8135 8112  432 8160 6169]
 48%|
Original indices: [8192 7808 7592 8216 8072 8336 8168 8119 7208 8335]
MMR indices: [8192 7808 7592 8216 8336 8072 8167 8335 8119 7400]
 52%|
Original indices: [8632 8608 8560 8536 8200 8631 8584 8559 8392 8512]
MMR indices: [8632 8608 8200 8631 4671 8559 8538 8584 8560 8536]
 55%|
Original indices: [6912 6865 7032 7200 7392 7416 7657 7296 6145 7272]
MMR indices: [6912 7657 1157 6865 8496 4418 7248 7200 7416 5521]
 58%|
Original indices: [7520 6058 7521 7519 4638 6059 6057 4637  971  972]
MMR indices: [7520 4638 6058  972  969 5843   36 5841 2439 7521]
 61%|
Original indices: [5752 6329 5728 5801 6497 5344 6664 6544 5584 5416]
MMR indices: [5752 7121 6017 6353 5344 8465 5801 4833 6664 6329]
 64%|
```

- The original indices (the top ones) are being returned by similarity search alone.
- The MMR indices show the final selected matches after applying relevance + diversity.
- Since the indices changed, MMR is actively replacing some similar/redundant matches with different retrieved time-series segments.
- This confirms the new retrieval logic is being used.

Results: Original vs MMR

MAE

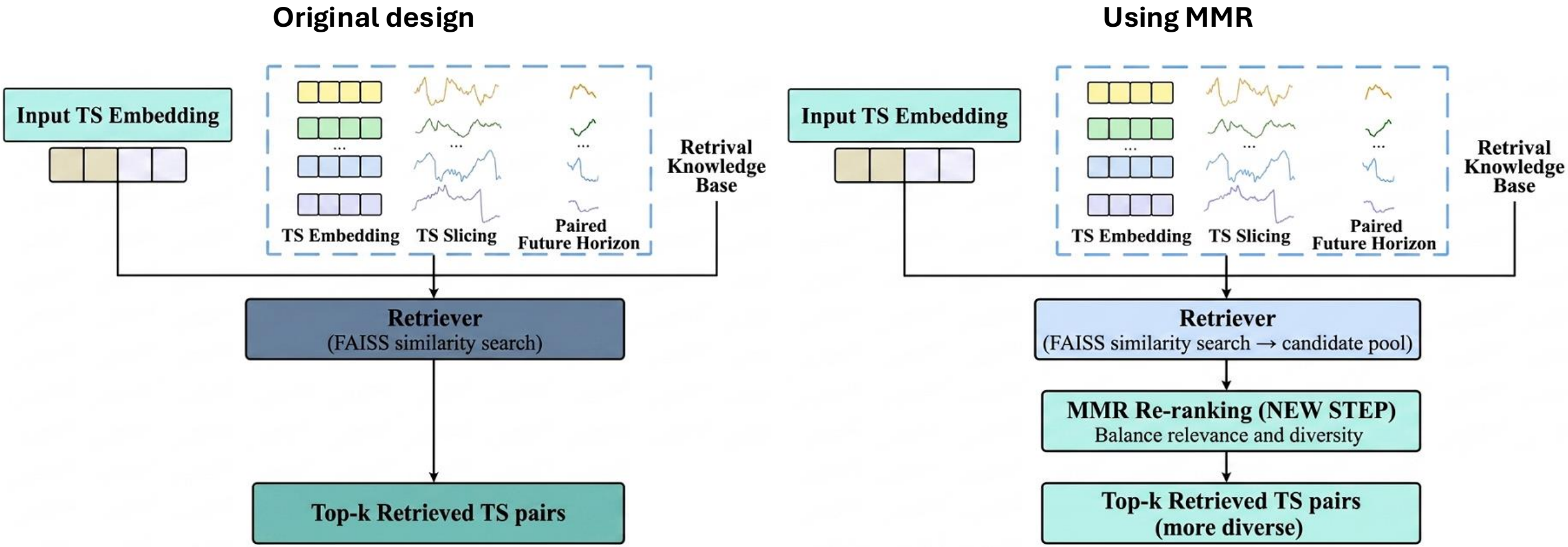
Dataset	Original	MMR
ETTh1	0.3556	0.3556
ETTh2	0.2451	0.2449

MSE

Dataset	Original	MMR
ETTh1	0.3623	0.3624
ETTh2	0.2981	0.2975

Although MMR altered the retrieved time-series segments, the resulting changes in MSE and MAE were very small, suggesting that retrieval diversity had **limited impact** on performance in this setup.

Architecture comparison



Future testing

What I would like to test in the future, if given the time

- Test MMR on more datasets, such as ETTm1, ETTm2, and Weather
- Try different MMR settings, such as changing the relevance/diversity weight
- Compare different top_k values, such as 5, 10, 20, or 64
- Test longer forecast horizons, such as 96, 192, 336, and 720

Sources

Identifying the issue, discusses redundancy
[1] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 9459–9474.[Online]. Available: https://arxiv.org/abs/2005.11401

Discusses why MMR works
[2] J. Carbonell and J. Goldstein, “The use of MMR, diversity-based reranking for reordering documents and producing summaries,” in Proc. 21st Annu. Int. ACM SIGIR Conf. Research and Development in Information Retrieval, 1998, pp. 335–336.

MMR pros/cons, some starter code, other methods
[3] A. Gupta, “Retriever in RAG,” Medium, Mar. 2024. [Online]. Available: https://medium.com/@ayushigupta9723/retriever-in-rag-4c25cd2cf2c0. [Accessed: Apr. 29, 2026]